IEEE Access
Multidisciplinary ⋮ Rapid Review ⋮ Open Access Journal

Draft for internal review, October 2026.

*Digital Object Identifier 10.1109/ACCESS.2024.0429000*

# What Drives Mamba-Based Sequential Recommenders? A Controlled Study of Selectivity, Recency, and the Last Item

**ABSTRACT** Mamba, a selective state space model, is increasingly used as the sequence encoder of next-item recommenders. Reported gains over self-attention and recurrent baselines are several percent, but the component that produces them has not been isolated. We report a controlled study in which all models share the embedding, normalization, feed-forward layers, loss, and training pipeline; only the sequence mixer changes. Starting from the selective Mamba mixer, we remove one ingredient at a time: input-dependent parameters, state expansion and gating, order information, and finally all mixing. All models receive the same tuning budget and are evaluated by full ranking over five seeds, with hypotheses fixed in advance. On three Amazon datasets, where the median user has six interactions, a fixed exponential recency filter is more accurate than the Mamba mixer on every dataset (4.3% to 6.7% higher ranking accuracy, 14 of 15 paired runs). An order-free mixer that sees only the most recent item and the history mean is non-inferior to Mamba. On MovieLens-1M, with a median of 96 interactions, the state space mixers are the most accurate, and their advantage over order-free pooling grows with history length. In neither setting does input dependence give a reliable gain over the same block with time-invariant parameters. Removing the most recent item costs Mamba about one third of its accuracy on the short-history datasets. The value of a Mamba encoder therefore depends on history length rather than on selectivity, and recency baselines should be part of every comparison.

**INDEX TERMS** Mamba, recommender systems, reproducibility, selective state space models, sequential recommendation, statistical hypothesis testing.

## I. INTRODUCTION

SEQUENTIAL recommenders predict the next item a user will interact with from the ordered history of past interactions [1]–[3]. Early models captured item-to-item transitions with Markov chains [4], [5]. Neural encoders based on recurrence [6], [7], convolution [8], and self-attention [9], [10] followed, and SASRec [9] has become the usual reference architecture. Since the introduction of selective state space models [11], a growing line of work replaces self-attention by Mamba blocks [12]–[16]. Two arguments are given for this choice. Inference time grows linearly with the history length, and the selection mechanism, through its input-dependent step size and projections, lets the model decide what to keep in its state [11].

The reported gains of Mamba-based recommenders over SASRec are moderate. Mamba4Rec, for example, reports gains of 5.6% to 8.0% over SASRec in the normalized discounted cumulative gain at rank 10 (NDCG@10) on three datasets, while its hit rate at rank 10 (HR@10) is lower than that of SASRec on two of them [12]. Differences of this size lie within the range that the loss function, tuning, and implementation details produce for the same baseline [17]–[19], and well-tuned simple methods have repeatedly matched newer neural models [20]–[22]. Benchmark datasets also differ in how sequential they are [23]. It is therefore not known which component of a Mamba recommender is responsible for its accuracy, or under which data conditions the selection mechanism matters.

We address this question with a controlled design (Fig. 1). All models share the item embedding, normalization, feed-forward sub-layers, residual connections, output layer, loss, and training pipeline; only the sequence mixer inside each block changes. Starting from the selective Mamba mixer, we remove one ingredient at a time. Making its step size and projections constant removes input dependence. Replacing the block by a per-channel exponential recency filter further removes state expansion and gating. A causal running mean removes order, although through the residual path it still sees the most recent item and the history mean. Removing all mixing leaves only the most recent item. Self-attention and

a recurrent network serve as references. All models receive the same tuning budget, results are averaged over five seeds and compared with paired statistics, and the hypotheses and decision rules were written down before the confirmatory runs.

The answer depends on the data. On three Amazon datasets, where the median user has six interactions, the exponential recency filter is the most accurate mixer, and the order-free mixer is non-inferior to Mamba. On MovieLens-1M, with a median of 96 interactions, the state space mixers are the most accurate. In neither case does the selection mechanism give a reliable gain over its time-invariant counterpart.

The contributions of this paper are as follows.

1) A controlled decomposition of Mamba-based sequential recommenders into selectivity, recurrent state, recency, order, and last-item components, with a matched wrapper, a matched tuning budget, five seeds, and pre-registered paired tests.
2) Evidence that on short e-commerce histories a fixed recency filter is more accurate than the Mamba mixer and an order-free mixer is non-inferior to it. On long histories, a recurrent state over the whole history is needed. Input-dependent selectivity gives no reliable gain in either regime.
3) An analysis of what the models use, based on test-time placebos, retraining on permuted histories, history length, and target popularity. On short histories, order matters mainly through the identity of the last item.
4) Robustness checks on embedding size and on a larger catalogue, an efficiency comparison, and a public release of the code, data splits, per-run results, and frozen protocol.

Section II reviews related work, Section III describes the controlled design, and Section IV the experimental protocol. Section V reports the results, Section VI discusses them, and Section VII concludes.

## II. RELATED WORK

### A. SEQUENTIAL RECOMMENDATION AND RECENCY

Markov-chain models predict the next item from the most recent one or a few items [4], [5]. Neural encoders use recurrence (GRU4Rec [6], [24], NARM [7]), convolution (Caser [8]), or self-attention (SASRec [9], BERT4Rec [10], TiSASRec [25]). Several models give the most recent interaction a dedicated role. STAMP [26], for instance, combines a representation of the last click with the mean of the session, the same information that our order-free "last + mean" mixer receives. Efficiency-oriented encoders replace full attention by low-rank or linear attention (LightSANs [27], LinRec [28]), learnable filters (FMLP-Rec [29]), or linear recurrences (LRURec [30], RecBLR [31]).

Recency has a long history as an explicit signal. Collaborative filtering has weighted interactions by their age [32] and modeled temporal drift [33], and recency is a strong predictor of repeat consumption [34]. In session-based recommendation, nearest-neighbor methods that weight the items of a session by their position or age [35], [36] are competitive with, and often better than, neural models [22], [37], [38]. Our exponential recency filter brings this inductive bias into the shared neural wrapper, so that it can be compared with the Mamba mixer under identical conditions.

### B. STATE SPACE MODELS AND RECOMMENDATION

Structured state space models [39], [40] process a sequence with a linear recurrence that can be computed as a convolution or a parallel scan. Diagonal state matrices [41], [42] and linear recurrent units [43] simplify this recurrence with little loss in accuracy. Mamba [11] makes the step size and the input and output projections functions of the input, which its authors call selectivity, and computes the resulting time-varying recurrence with a hardware-aware scan. Mamba-2 [44] relates this recurrence to attention; Qu *et al.* [45] survey Mamba and its applications. Related linear-time models differ mainly in whether the state decays at a fixed or an input-dependent rate [46], [47]. Exponential moving averages have also served as an explicit inductive bias in attention models [48].

In recommendation, Mamba4Rec [12] replaces the self-attention sub-layer of SASRec by a Mamba block. Later models add gating and bidirectionality (SIGMA [13]), use the state space duality formulation (SSD4Rec [14]), add time awareness (TiM4Rec [15]), or add local-context and consistency modules [16]. SIGMA attributes part of its design to the observation that Mamba's state estimation is unstable for short-term patterns and adds a separate GRU for short sequences [13]. The ablation studies of these models remove whole modules. To our knowledge, none of them compares the selective mixer with the same block with time-invariant parameters or with a fixed recency filter under a matched wrapper.

Outside recommendation, several studies ask which ingredients of Mamba matter. In language modeling, input dependence matters for associative recall and copying [49], [50], Mamba layers can be read as implicit attention [51], and they show recency effects [52]. In vision, the forget gate and the block design account for much of Mamba's accuracy [53]. A model that keeps the Mamba block but drops its state space mixer even matches visual Mamba models on image classification [54]. Mamba-based models have also been examined against Transformer baselines for time-series forecasting [55]. Sieber *et al.* [56] compare attention, state space models, and recurrent networks in one dynamical-systems framework. We ask the same kind of question for sequential recommendation, where many benchmark histories are short and sparse.

### C. RIGOR IN RECOMMENDER EVALUATION

Reproducibility studies have found that many neural recommenders do not outperform well-tuned simple baselines [20], [21], [57], including in session-based recommendation [22], [58], and have called for shared protocols and released artifacts [59], [60]. For sequential models, the training loss explains much of the reported gap between BERT4Rec and

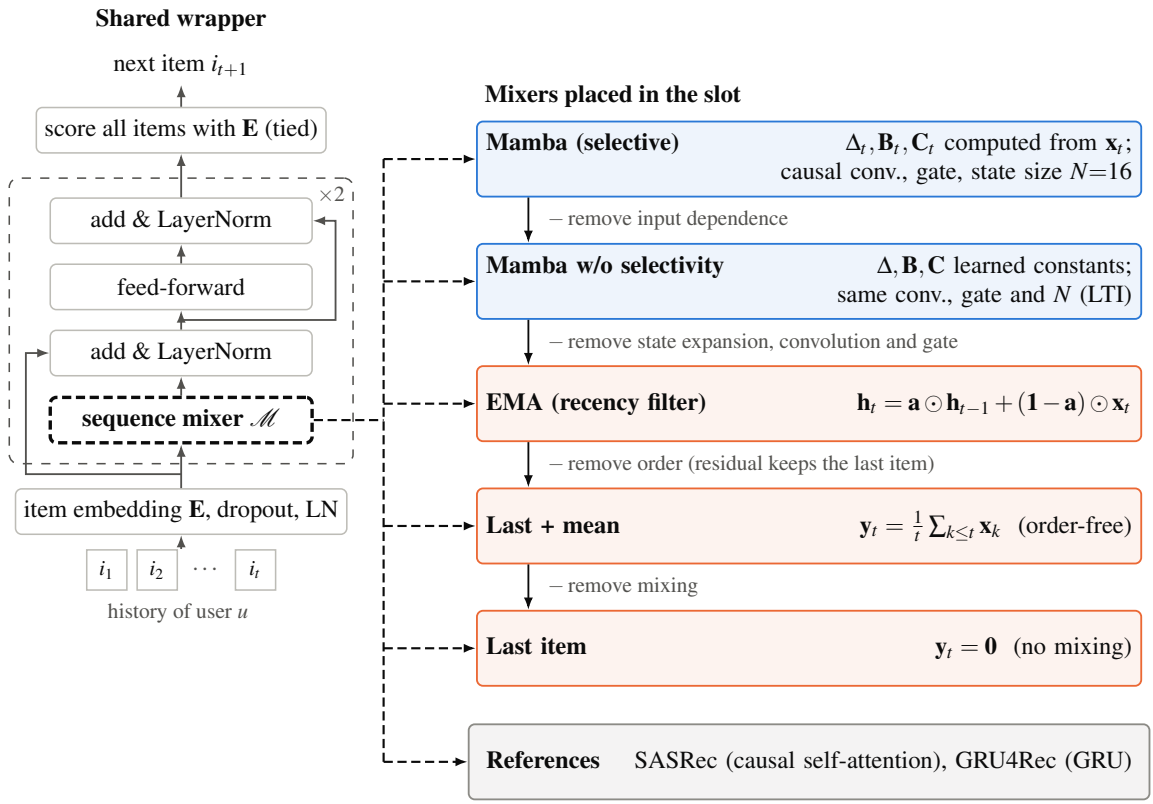



**FIGURE 1. Controlled design. Every model uses the same wrapper (left); only the sequence mixer $\mathcal{M}$ in the two blocks changes. The mixers on the right are ordered so that each step removes one ingredient of the selective Mamba mixer. SASRec and GRU4Rec are included as reference models (GRU4Rec uses the same embedding, output layer, and loss, but not the block structure). LN: layer normalization; LTI: linear time-invariant.**

SASRec [17], [18], [61], and third-party implementations of the same model can differ substantially in accuracy [19].

The evaluation setting matters as well. Sampled metrics can change model rankings [62], [63], and the choice of data split affects both the results and the leakage of future information [64], [65]. Work on statistical practice in information retrieval and machine learning recommends effect sizes with confidence intervals, corrections for multiple comparisons, and accounting for variation between random seeds [66]–[69]. Klenitskiy *et al.* [23] shuffle user sequences to measure how sequential a dataset is. We use shuffling as one of two test-time placebos and as a training condition, but our aim is different: to attribute the accuracy of one architecture to its components.

## III. CONTROLLED DESIGN

### A. PROBLEM AND SHARED WRAPPER

Let $s_u = (i_1, \ldots, i_t)$ be the chronologically ordered history of user $u$ over the item set $\mathcal{I}$. A model outputs a score for every item in $\mathcal{I}$ for position $t+1$. All models share the wrapper

$$\begin{aligned} \mathbf{X}^{(0)} &= \mathrm{LN}\big(\mathrm{Drop}(\mathbf{E}[s_u])\big), \\ \mathbf{Z}^{(b)} &= \mathrm{LN}\big(\mathbf{X}^{(b)} + \mathrm{Drop}(\mathcal{M}^{(b)}(\mathbf{X}^{(b)}))\big), \\ \mathbf{X}^{(b+1)} &= \mathrm{LN}\big(\mathbf{Z}^{(b)} + \mathrm{Drop}(\mathrm{FFN}(\mathbf{Z}^{(b)}))\big), \quad b = 0, 1, \end{aligned} \tag{1}$$

where $\mathbf{E} \in \mathbb{R}^{|\mathcal{I}| \times d}$ is the item embedding table and LN is layer normalization [70]. FFN is a two-layer feed-forward network with Gaussian error linear unit (GELU) activation and inner size $4d$, and $\mathcal{M}^{(b)}$ is the sequence mixer of block $b$. The representation $\mathbf{h}_t$ at the last position is scored against all items with the tied embedding table, $\hat{\mathbf{y}} = \mathbf{E}\mathbf{h}_t$. Training uses full-softmax cross-entropy at every position of the sequence, which makes SASRec-style models competitive [17]. Because of the residual connection, the representation of the current item always reaches the output; the mixer adds information from earlier positions.

### B. MIXERS

Each mixer maps a sequence $\mathbf{x}_1, \ldots, \mathbf{x}_t \in \mathbb{R}^d$ to outputs $\mathbf{y}_1, \ldots, \mathbf{y}_t$ and uses only positions up to its own.

*Mamba (selective).* We use the Mamba block [11] as in Mamba4Rec [12]. A linear projection maps the input to $2d$ channels and to a gate branch. A causal depth-wise convolution of width 4 with sigmoid linear unit (SiLU) activation produces $\mathbf{u}_t$, and every channel $c$ keeps a state $\mathbf{h}_{t,c} \in \mathbb{R}^N$ updated by

$$\begin{aligned} \mathbf{h}_{t,c} &= e^{\Delta_{t,c}\mathbf{A}_c}\, \mathbf{h}_{t-1,c} + \Delta_{t,c}\, \mathbf{B}_t\, u_{t,c}, \\ y_{t,c} &= \mathbf{C}_t^{\top} \mathbf{h}_{t,c} + D_c\, u_{t,c}, \end{aligned} \tag{2}$$

where $\mathbf{A}_c$ is diagonal and initialized as $-(1, 2, \ldots, N)$ [41]. The step size $\Delta_t$ and the projections $\mathbf{B}_t, \mathbf{C}_t \in \mathbb{R}^N$ are

linear functions of $\mathbf{u}_t$; the step size passes through a low-rank projection and a softplus. The output is multiplied by the SiLU-activated gate branch and projected back to $d$ channels. The state size is $N = 16$.

*Mamba without selectivity.* The same block, but $\Delta$ (one value per channel), $\mathbf{B}$, and $\mathbf{C}$ are learned constants. Equation (2) then describes a linear time-invariant system with the same projections, convolution, gate, and state size. The difference between this mixer and Mamba isolates input dependence.

*EMA (recency filter).* An exponential moving average (EMA) computes $\mathbf{y}_t = \mathbf{a} \odot \mathbf{y}_{t-1} + (\mathbf{1} - \mathbf{a}) \odot \mathbf{x}_t$ with a learned per-channel decay $\mathbf{a} = \sigma(\mathbf{w}) \in (0, 1)^d$. It has $d$ parameters per block and no state expansion, convolution, gate, or input dependence. It is a time-invariant recurrence with a one-dimensional state per channel.

*Last + mean.* The causal running mean $\mathbf{y}_t = \frac{1}{t} \sum_{k \le t} \mathbf{x}_k$. It ignores order; together with the residual connection, the block sees the most recent item and the mean of the history.

*Last item.* $\mathcal{M} \equiv \mathbf{0}$, so that only the most recent item reaches the output, a neural counterpart of a first-order Markov model [4].

*References.* SASRec uses causal multi-head self-attention with two heads and learned positional embeddings [9]. GRU4Rec uses one GRU layer on the embedded sequence [6] followed by the same output layer and loss; it does not use the block structure of (1).

All models use $d = 64$ and two blocks; Mamba4Rec uses one Mamba layer by default [12], and we use two so that every mixer has the depth of SASRec. The selective scan uses the official `mamba_ssm` CUDA kernel, which we checked against a direct implementation of the recurrence (largest absolute difference $1.1 \times 10^{-5}$).

### C. WHAT EACH COMPARISON ISOLATES

Adjacent mixers in Fig. 1 differ in one ingredient. Mamba versus Mamba without selectivity isolates input dependence. Mamba without selectivity versus EMA isolates the expanded state, the convolution, and the gate. EMA versus last + mean isolates order beyond the identity of the most recent item, and last + mean versus last item isolates the content of the earlier history. Each comparison with the selective Mamba mixer shows how much of its accuracy a simpler mixer retains.

## IV. EXPERIMENTAL PROTOCOL

### A. DATASETS

Table 1 summarizes the data. Three Amazon review categories [71], [72] (Baby; Sports and Outdoors; Clothing, Shoes and Jewelry) provide short sequences; we use the 5-core form distributed with the MMRec benchmark [73]. MovieLens-1M (ML-1M) [74] provides long sequences, and Amazon Electronics (5-core) serves as a scale check. Every rating is treated as an implicit interaction. Interactions are split leave-one-out by time [64]: the last interaction of each user is the test target and the second last the validation target. The Amazon timestamps have a resolution of one day, and interactions with equal timestamps keep the order of the source files. For testing, the validation item is appended to the history. Items already in a user's history are excluded from ranking. The maximum input length is 50 for the Amazon datasets and 200 for MovieLens-1M.

**TABLE 1. Dataset statistics. Sequence length counts all interactions of a user, including the validation and test items.**

| Dataset | Users | Items | Interact. | Mean len. | Median len. |
|---|---|---|---|---|---|
| Baby | 19,445 | 7,050 | 160,792 | 8.3 | 6 |
| Sports | 35,598 | 18,357 | 296,337 | 8.3 | 6 |
| Clothing | 39,387 | 23,033 | 278,677 | 7.1 | 6 |
| MovieLens-1M | 6,040 | 3,706 | 1,000,209 | 165.6 | 96 |
| Electronics | 192,403 | 63,001 | 1,689,188 | 8.8 | 7 |

### B. TRAINING AND TUNING

All models are trained with Adam [75]. In each epoch, a user with at least two training interactions contributes their most recent $L + 1$ training interactions, where $L$ is the maximum input length. The batch size is 256 (128 for MovieLens-1M and Electronics; see Appendix A). Training runs for at most 200 epochs, with validation every two epochs and early stopping after five evaluations without improvement; the best validation checkpoint is kept. Every model receives the same grid on every dataset: learning rate $\{10^{-3}, 3 \times 10^{-3}\}$ by dropout $\{0.2, 0.5\}$, evaluated with seed 0 and selected by NDCG@10 on the validation set. The final runs use seeds 0–4 with the selected configuration. Electronics reuses the configurations selected on Sports and uses seed 0 only. For large catalogues, the loss is computed in chunks of positions, which leaves the gradients unchanged. Tables 8 and 9 in Appendix C list all fixed settings and the configuration selected for every model and dataset.

### C. EVALUATION

Each test user's target is ranked against all items. Ties are broken pessimistically: a target that ties with $k$ other items is ranked below all of them. Full ranking avoids the distortions of sampled metrics [62], [63]. We report NDCG@10 (primary) and HR@10. Results at cut-off 20 are part of the released files. At that cut-off, the recency filter remains ahead of Mamba on the three Amazon datasets, and Mamba remains the most accurate mixer on MovieLens-1M. The rank of every test user is stored, so every stratification in this paper can be recomputed.

### D. PLACEBOS

Without retraining, we evaluate each model with (i) the test history randomly permuted and (ii) the most recent item removed from the test history. A model that does not use order is unaffected by (i); (ii) measures how much a model relies on the last interaction.

### E. HYPOTHESES AND STATISTICS

The protocol, including five hypotheses and their decision rules, was fixed before the confirmatory runs (Appendix A).

H1: on the pooled Amazon datasets, the upper bound of the 95% interval for Mamba versus Mamba without selectivity is below 3%. H2: on the pooled Amazon datasets, the lower bound of the interval for last + mean versus Mamba is above $-3\%$, i.e., last + mean is non-inferior with a 3% margin [76]. H3: removing the most recent item reduces Mamba's NDCG@10 by at least 20% on every Amazon dataset. H4: on MovieLens-1M, Mamba and SASRec each exceed last + mean by at least 5%, with interval lower bounds above zero. H5: on MovieLens-1M, the interval for Mamba versus Mamba without selectivity lies above zero.

Comparisons are paired over (dataset, seed). For a metric value $m$ of models A and B, we use the relative difference $m_{\mathrm{A}}/m_{\mathrm{B}} - 1$ and report its mean with a 95% percentile bootstrap interval (20,000 resamples) [77]. We apply exact Wilcoxon signed-rank tests [78] with Holm correction [79] over the pre-registered family [67], [80]. Pairing by seed and reporting intervals instead of single runs follows recommendations on variance in machine learning benchmarks [68], [69]. With five pairs, the smallest attainable two-sided Wilcoxon $p$-value is 0.0625, so decisions for single datasets rest on the intervals, as specified in the protocol. H2 was suggested by a pilot study on two Amazon datasets with two seeds; the confirmatory runs use new tuning, new seeds, and additional datasets. Fixing hypotheses in advance separates confirmatory from exploratory results [81], and analyses added after the protocol are labeled as such.

### F. IMPLEMENTATION CHECK

Mamba4Rec [12], implemented in RecBole [82], reports results on MovieLens-1M with input length 200 and $d = 64$: NDCG@10 of 0.1822 for Mamba4Rec, 0.1687 for SASRec, and 0.1642 for GRU4Rec (HR@10 0.3121, 0.2977, and 0.2934). Our implementations reach 0.1875, 0.1666, and 0.1599 (HR@10 0.3223, 0.2930, and 0.2804). The preprocessing differs slightly (3,706 versus 3,416 items), so the values are not identical, but our Mamba mixer and the reference models reach the accuracy level of the original study.

### G. REPRODUCIBILITY AND CODE

The code, data splits, per-run results, and analysis scripts are available at https://github.com/austinukm/MSR-CS and in the supplementary material. The repository has four parts:

- *Code*: the shared wrapper, all mixers, training, evaluation, and the placebos.
- *Data*: the leave-one-out splits of the four Amazon datasets, a script that recreates the MovieLens-1M split from the official archive, and a script that checks every split against its SHA-256 hash and prints the statistics of Table 1.
- *Results*: one file per run with the configuration, validation and test metrics, per-user test ranks, strata, placebo results, and timing.
- *Analysis*: scripts that recompute every table, figure, and number in this paper from the result files, without retraining.

The frozen protocol and its English translation are included. Appendix C lists all settings and maps these artifacts to a common reproducibility checklist [83].

The study comprises 333 training runs: 112 for tuning, 146 final runs, 45 with permuted training histories, and 30 with $d = 128$. They took about 9 hours of training computation, excluding validation passes and the pauses described in Appendix A. All runs used one NVIDIA GeForce RTX 4060 Laptop graphics processing unit (GPU) with 8 GB of memory, Python 3.13, PyTorch 2.11 with CUDA 13.0, `mamba_ssm` 2.3.2, and NumPy 2.4. Each run sets the seeds of PyTorch and NumPy. GPU kernels are not bit-wise deterministic, so a repeated run can differ slightly from the stored one; the variation over five seeds bounds this effect.

## V. RESULTS

### A. MAIN COMPARISON

Table 2 reports test accuracy over five seeds, and Fig. 2 shows the paired differences to the selective Mamba mixer. Two regimes appear. On the three Amazon datasets, the exponential recency filter is the most accurate mixer and exceeds Mamba in 14 of 15 paired runs. Its mean advantage is 6.4% on Baby (95% interval [3.0, 9.3]), 6.7% on Sports ([4.6, 9.1]), and 4.3% on Clothing ([1.3, 6.7]). The last + mean mixer has no parameters and uses order only through the residual path of the current item, yet it is within 1.5% of Mamba on all three datasets. SASRec and GRU4Rec are less accurate than every state space or recency mixer. The two state space mixers do not form a consistent order: the time-invariant version is ahead on Baby ($+3.2\%$) and behind on Sports ($-5.1\%$) and Clothing ($-4.3\%$).

On MovieLens-1M the order reverses. The two state space mixers are the most accurate and statistically indistinguishable ($-0.9\%$, $[-2.5, +0.8]$), followed by the recency filter ($-7.6\%$), SASRec ($-11.2\%$), GRU4Rec ($-14.8\%$), last + mean ($-21.2\%$), and the last item alone ($-33.5\%$).

### B. PRE-REGISTERED HYPOTHESES

Table 3 lists the five pre-registered tests; three hold and two do not. H2 holds: last + mean is non-inferior to Mamba on the pooled Amazon datasets ($+0.67\%$, $[-0.62, +1.92]$). H3 holds: removing the most recent item reduces Mamba's NDCG@10 by 30.8%, 33.7%, and 33.0% on Baby, Sports, and Clothing. H4 holds: on MovieLens-1M, Mamba and SASRec exceed last + mean by 27.0% and 12.8%. H1 does not hold as stated. The pooled gain of selectivity on Amazon is $+2.33\%$, and its interval $[-0.09, +4.58]$ extends above the 3% bound, so a gain of up to 4.6% cannot be excluded. The gain is also not significantly different from zero, and its sign varies by dataset. H5 does not hold either: on MovieLens-1M, the selective mixer is $0.92\%$ ahead of its time-invariant version, with an interval that includes zero.

### C. WHAT THE MODELS USE

*Test-time placebos.* For every mixer and dataset, Fig. 3(a) plots the loss of NDCG@10 under permutation against the

**TABLE 2.** Test accuracy (mean over five seeds; standard deviation of NDCG@10 in small type). Best value per column in bold, second best underlined.

| | Baby | | Sports | | Clothing | | MovieLens-1M | |
|---|---|---|---|---|---|---|---|---|
| Mixer | NDCG@10 | HR@10 | NDCG@10 | HR@10 | NDCG@10 | HR@10 | NDCG@10 | HR@10 |
| Last item | .0253±.0007 | .0437 | .0250±.0004 | .0414 | .0137±.0002 | .0231 | .1247±.0007 | .2149 |
| Last + mean | .0267±.0005 | .0479 | <u>.0274</u>±.0005 | <u>.0489</u> | <u>.0160</u>±.0003 | **.0289** | .1477±.0014 | .2587 |
| EMA (recency) | **.0286**±.0005 | **.0506** | **.0289**±.0006 | **.0495** | **.0164**±.0006 | <u>.0288</u> | .1733±.0016 | .2992 |
| Mamba w/o selectivity | <u>.0278</u>±.0004 | <u>.0494</u> | .0257±.0005 | .0436 | .0150±.0005 | .0265 | <u>.1859</u>±.0024 | <u>.3184</u> |
| Mamba (selective) | .0269±.0008 | .0478 | .0271±.0004 | .0469 | .0157±.0002 | .0278 | **.1875**±.0021 | **.3223** |
| SASRec | .0256±.0005 | .0456 | .0254±.0005 | .0428 | .0130±.0002 | .0224 | .1666±.0012 | .2930 |
| GRU4Rec | .0233±.0007 | .0436 | .0196±.0005 | .0339 | .0096±.0004 | .0172 | .1599±.0060 | .2804 |

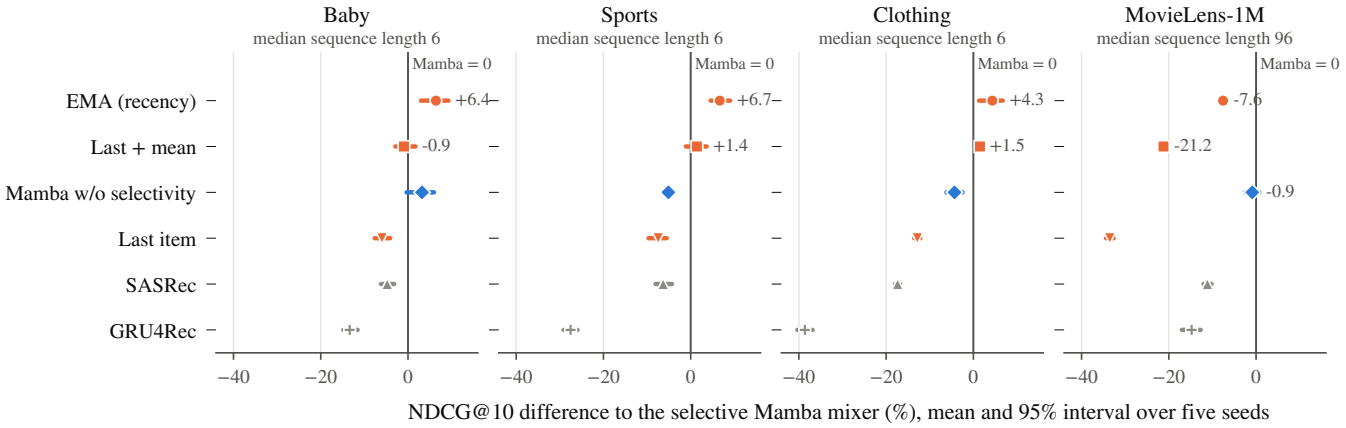



**FIGURE 2.** Paired NDCG@10 difference of every mixer to the selective Mamba mixer (marker: mean over five seeds; bar: 95% bootstrap interval). Points to the right of the vertical line are more accurate than Mamba. Orange: recency and pooling mixers; blue: state space mixers; grey: reference models.

**TABLE 3.** Pre-registered tests. Relative NDCG@10 difference of the first model to the second, paired over (dataset, seed), with 95% bootstrap interval, number of pairs in which the first model wins, exact Wilcoxon *p*-value, and Holm-corrected *p*-value.

| Test | Comparison | Data | Diff. | 95% interval | Wins | $p$ | $p_{\text{Holm}}$ | Decision rule and outcome |
|---|---|---|---|---|---|---|---|---|
| H1 | Mamba vs. Mamba w/o selectivity | Amazon | +2.33% | [−0.09, +4.58] | 11/15 | 0.23 | 0.69 | upper bound $<3\%$: not met |
| H2 | Last + mean vs. Mamba | Amazon | +0.67% | [−0.62, +1.92] | 9/15 | 0.45 | 0.88 | lower bound $>-3\%$: met |
| H4a | Mamba vs. last + mean | ML-1M | +26.96% | [+25.28, +28.42] | 5/5 | 0.062 | 0.31 | $\geq 5\%$, lower bound $>0$: met |
| H4b | SASRec vs. last + mean | ML-1M | +12.78% | [+12.23, +13.32] | 5/5 | 0.062 | 0.31 | $\geq 5\%$, lower bound $>0$: met |
| H5 | Mamba vs. Mamba w/o selectivity | ML-1M | +0.92% | [−0.75, +2.51] | 3/5 | 0.44 | 0.88 | lower bound $>0$: not met |

Amazon: Baby, Sports, and Clothing pooled (15 pairs). H3, a threshold test on each dataset, is reported in the text. With five pairs, the smallest attainable two-sided Wilcoxon *p*-value is 0.0625, so the decisions for MovieLens-1M rest on the intervals.

loss without the most recent item; Table 7 in Appendix B gives the values. On the Amazon datasets, the recurrent, state space, and attention mixers lose 28%–40% under permutation and 30%–37% without the last item. The two losses are of similar size, which indicates that order matters on these datasets mainly through *which item is last*. Last + mean loses least under permutation (18%–27%) because only its residual path depends on order. On MovieLens-1M, permutation is far more damaging (76%–83%) than removing the last item (20%–30%): here the models use the order of the whole history.

*Training on permuted histories.* The placebo losses are measured on models trained with ordered histories, so part of them could reflect a mismatch between training and test inputs rather than the use of order. We therefore retrained the five main mixers on the Amazon datasets with every training sequence randomly permuted in every epoch (seeds 0–2, configurations from the main tuning). Table 4 evaluates them on the ordered test histories. Training without order costs every mixer 31%–42% of NDCG@10 on Baby, 21%–34% on Sports, and 6%–19% on Clothing. The ordered models do learn from order, consistent with the last-item placebo. The ranking does not change: with permuted training, the recency filter and last + mean remain ahead of both state space mixers and SASRec on all three datasets.

### D. HISTORY LENGTH

Fig. 3(b) splits the test users into tertiles of history length. On MovieLens-1M, the advantage of the state space mixers over last + mean grows with length: for Mamba it is 18% in the shortest tertile, 30% in the middle one, and 41% in the longest (NDCG@10 of 0.2316 versus 0.1968, 0.1856

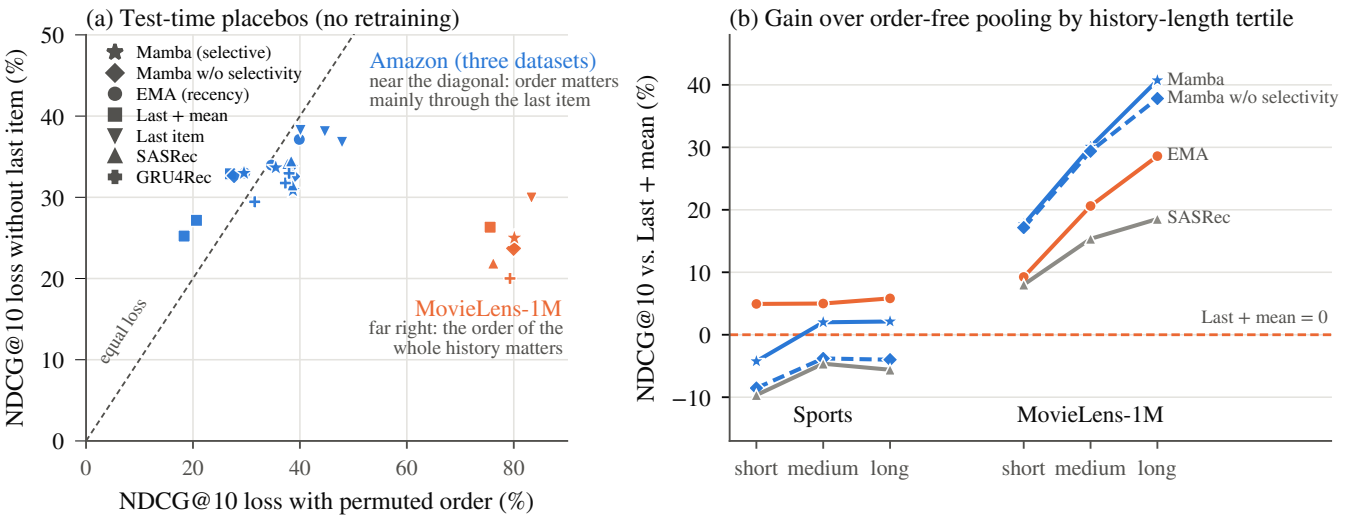



**FIGURE 3.** **What the models use. (a) Loss of test NDCG@10 when the test history is permuted (horizontal) or its most recent item is removed (vertical), for every mixer on the three Amazon datasets (blue) and MovieLens-1M (orange); no retraining. (b) NDCG@10 of each mixer relative to last + mean within history-length tertiles of the test users (Sports: up to 5, 6–7, and more than 7 items; MovieLens-1M: up to 56, 57–155, and more than 155 items).**

**TABLE 4.** **Training on permuted histories. Test NDCG@10 of the models trained on ordered histories (ord.) and relative change (%) when the same models are trained on randomly permuted histories (perm.); both are evaluated on the ordered test histories, mean over seeds 0–2.**

| | Baby | | Sports | | Clothing | |
|---|---|---|---|---|---|---|
| Mixer | ord. | perm. | ord. | perm. | ord. | perm. |
| Last + mean | .0266 | −31 | .0274 | −21 | .0161 | −9 |
| EMA (recency) | .0287 | −33 | .0291 | −22 | .0166 | −17 |
| Mamba w/o sel. | .0276 | −40 | .0256 | −32 | .0149 | −10 |
| Mamba (selective) | .0273 | −42 | .0269 | −34 | .0158 | −19 |
| SASRec | .0260 | −38 | .0251 | −34 | .0130 | −6 |

versus 0.1427, and 0.1446 versus 0.1028). The recency filter and SASRec follow the same trend at a lower level. On the Amazon datasets, the tertile boundaries lie between four and seven items. There, the recency filter is ahead of Mamba in every tertile except the longest one on Clothing (−0.1%). Its advantage is largest for the shortest histories (+6.9%, +9.6%, and +9.5% on Baby, Sports, and Clothing) and smaller for the longest (+3.5%, +3.6%, and −0.1%). The direction matches MovieLens-1M: the longer the history, the more the state space mixer gains relative to pooling and recency.

### E. TARGET POPULARITY

We grouped test targets by their number of training interactions: at most 5 (cold), 6–20 (mid), and more than 20 (head). On Sports and Clothing, the advantage of the recency filter over Mamba comes from head targets (+16%, [10, 22], and +11%, [5, 16]). On cold targets, the recency filter is behind (−28%, [−51, +11], and −17%, [−21, −10]), although the first interval includes zero. On Baby, the recency filter is ahead in all three groups. Last + mean shows the same pattern on Sports and Clothing, with intervals that exclude zero in both groups. Pooled over the Amazon datasets, the recency filter gains 9.4% ([5.5, 13.6]) on head targets, which account for 25%–52% of the test users. The cold-target differences are large in relative terms but uncertain. Pooling the history thus mainly helps for popular targets. On rare targets, the selective mixer is more accurate than last + mean on Sports and Clothing, but these targets contribute little to the overall metric.

### F. ROBUSTNESS

*Embedding size.* Fig. 4 repeats the comparison with $d = 128$ on Sports and MovieLens-1M (seeds 0–2, configurations from the $d = 64$ tuning). On Sports, the results keep their direction. The recency filter and last + mean remain ahead of the selective mixer (+8.9% and +5.5%, compared with +6.6% and +1.4% at $d = 64$), and the selective mixer gains nothing from the larger embedding (+0.2%). On MovieLens-1M, the picture changes. The selective mixer loses 5.5% at $d = 128$, and the recency filter (+1.1%) and the time-invariant mixer (+1.9%) now match or slightly exceed it. Last + mean (−15.0%) and SASRec (−5.5%) stay behind. The advantage of recurrent state over order-free pooling on long histories thus holds at both sizes. The advantage of Mamba over the recency filter on MovieLens-1M, however, holds only at $d = 64$, and at $d = 128$ selectivity again gives no gain.

*Larger catalogue.* Table 5 reports Amazon Electronics (192k users, 63k items). With one run per mixer (seed 0), the differences are indicative only, but the pattern of the other Amazon datasets reappears. Relative to the selective mixer, the recency filter reaches +3.3% and last + mean +1.5%, whereas the time-invariant mixer (−4.6%), SASRec (−6.0%), and the last item alone (−9.9%) fall behind.

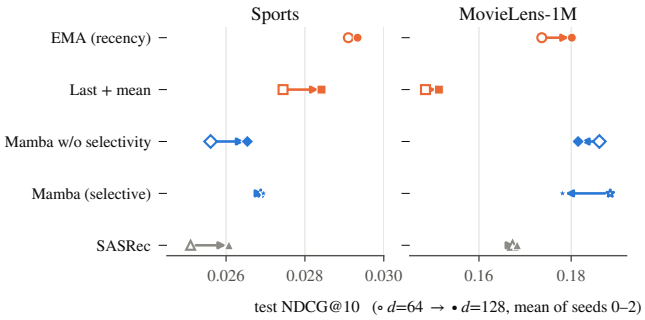



**FIGURE 4.** Effect of the embedding size. Test NDCG@10 at $d = 64$ (open marker) and $d = 128$ (filled marker), mean over seeds 0–2. On MovieLens-1M the selective mixer loses accuracy at $d = 128$, and the recency filter and the time-invariant mixer reach it.

**TABLE 5.** Scale check on Amazon Electronics (seed 0). Best value per column in bold; the last column is the NDCG@10 difference to the selective mixer.

| Mixer | NDCG@10 | HR@10 | vs. Mamba |
|---|---|---|---|
| Last item | .0246 | .0416 | −9.9% |
| Last + mean | .0277 | **.0488** | +1.5% |
| EMA (recency) | **.0282** | .0488 | +3.3% |
| Mamba w/o selectivity | .0261 | .0452 | −4.6% |
| Mamba (selective) | .0273 | .0476 | — |
| SASRec | .0257 | .0445 | −6.0% |

### G. EFFICIENCY

Table 6 lists model size and time. The selective mixer adds 66k parameters to a model whose embedding table holds 1.17M parameters on Sports; the recency filter adds 128. In our implementation, training and inference times of the state space and recency mixers are of the same order. The recency filter is the slowest mixer on MovieLens-1M only because we implemented it as a loop over positions, whereas the state space mixers use the fused CUDA kernel. An exponential filter is a linear time-invariant recurrence with a one-dimensional state and can use the same parallel scan. The difference is a property of our implementation, not of the model.

## VI. DISCUSSION

### A. WHAT DRIVES MAMBA-BASED RECOMMENDERS

The decomposition separates three ingredients that are usually introduced together. The input-dependent selection mechanism, the defining feature of Mamba, contributes least. On neither short nor long histories is the selective mixer reliably more accurate than the identical block with fixed parameters, and the direction of the difference changes across datasets. Recurrent state over the whole history matters on long, ordered histories. There, the state space mixers exceed attention and order-free pooling by wide margins, and the advantage grows with history length. On short e-commerce histories, recency matters most: an exponential filter with one decay rate per channel is the most accurate mixer, and an order-free mixer that sees the last item and the history mean is as accurate as Mamba.

**TABLE 6.** Parameters, training time per epoch, and inference time per user (full-catalogue scoring on the GPU), mean over seeds.

| | Sports | | | MovieLens-1M | | |
|---|---|---|---|---|---|---|
| Mixer | Param. | s/ep. | ms/user | Param. | s/ep. | ms/user |
| Last item | 1.242M | 1.22 | .006 | 0.304M | 0.78 | .023 |
| Last + mean | 1.242M | 1.26 | .006 | 0.304M | 0.81 | .027 |
| EMA | 1.242M | 2.03 | .022 | 0.304M | 5.08 | .099 |
| w/o sel. | 1.297M | 2.17 | .020 | 0.360M | 1.27 | .063 |
| Mamba | 1.308M | 2.26 | .020 | 0.370M | 1.36 | .071 |
| SASRec | 1.278M | 3.17 | .012 | 0.350M | 1.27 | .063 |
| GRU4Rec | 1.200M | 1.29 | .006 | 0.262M | 1.18 | .021 |

### B. WHY A RECENCY FILTER SUFFICES ON SHORT HISTORIES

With about six interactions per user, an expanded state has little to select from, while the additional projections must be estimated from sparse data. The placebos locate the useful signal: removing the most recent item costs every model about one third of its accuracy, and permuting the history costs about the same, because permutation mostly changes which item is last. A filter that weights recent items more heavily than older ones encodes exactly this structure. The same bias may explain why recency-weighted neighborhood methods are strong in session-based recommendation [35]–[37]. The popularity analysis adds that pooling the history mainly helps for popular targets, for which the history mean is informative. For rare targets on two datasets, the selective mixer is more accurate than last + mean. On long histories, by contrast, the order of the whole history carries information: permutation removes about 80% of the accuracy. At $d = 64$, a multi-dimensional recurrent state captures this information better than a single decay rate per channel.

### C. IMPLICATIONS

For model design, the choice of sequence encoder should follow the length and sequential structure of the target histories, which can be measured with the placebos used here or with dataset-level tests [23]. One trained baseline is enough for the placebos: for the Mamba models in this study, the ratio of the loss under permutation to the loss without the last item was between 0.9 and 1.3 on the four Amazon datasets and 3.2 on MovieLens-1M. A ratio near one indicates that the order information is concentrated in the last item, a setting in which a recency baseline is hard to beat. For evaluation, new sequential recommenders, whether based on Mamba or not, should be compared with a recency filter and a last + mean mixer under the same wrapper and tuning budget [59]. On short-history benchmarks, these baselines are strong. For claims about selectivity, the appropriate control is the same block with time-invariant parameters. The evidence should come from paired comparisons over several seeds, because the effects in question are of the same size as the variation between seeds and implementations.

## VII. CONCLUSION

Under a matched wrapper, tuning budget, and paired statistics, the accuracy of Mamba-based sequential recommenders is not explained by the selection mechanism. On short e-commerce histories, a fixed exponential recency filter is more accurate than the Mamba mixer and an order-free mixer is non-inferior to it. On long, ordered histories, recurrent state is more accurate than order-free pooling, with or without selectivity, and with a larger embedding a recency filter closes the gap to Mamba. The choice of encoder for sequential recommendation should start from the length and sequential structure of the histories, and recency baselines belong in every comparison.

## DATA AND CODE AVAILABILITY

All datasets are public [71], [73], [74]. The code, the Amazon splits (user and item identifiers, ratings, and timestamps only), the MovieLens-1M preprocessing script, all per-run results, the frozen protocol, and the analysis scripts are available at https://github.com/austinukm/MSR-CS and in the supplementary material (Section IV-G).

## APPENDIX A FROZEN PROTOCOL AND DEVIATIONS

The protocol was written before any confirmatory run. It is summarized here; the original file and an English translation are in the code repository and the supplementary material.

*Research questions.* RQ1: the gain of input-dependent selectivity over the time-invariant block. RQ2: how much accuracy is explained by a fixed recency filter, by the last item, and by the last item plus the history mean. RQ3: reliance on order and on the most recent item (two test-time placebos), stratified by history length and target popularity. RQ4: the boundary condition on long, sequential histories (MovieLens-1M). RQ5: efficiency.

*Design.* Data, wrapper, mixers, tuning grid, five final seeds, metrics, and placebos as described in Sections III and IV; Electronics reuses the Sports configurations. Hypotheses H1–H5 and the statistical procedure are given in Section IV.

*Deviation 1 (stability).* After two system crashes shortly after MovieLens-1M training began, the batch size for MovieLens-1M and Electronics was reduced from 256 to 128 for all models. The only MovieLens-1M run completed with batch size 256 was discarded and repeated. During parts of the remaining runs, we used scheduling measures that affect only wall-clock time: a pause after each batch and, for some runs, a wait for the GPU to cool down before the run started. They do not change any computation, seed, hyper-parameter, or evaluation, and the reported training times exclude the pauses. The protocol file lists the dated entries.

*Deviation 2 (scale check).* Electronics runs took about one hour each. The scale check was reduced from seven mixers and three seeds to six mixers (all except GRU4Rec) with seed 0, and the loss was computed in chunks of positions. Electronics enters no hypothesis.

*Additions after the protocol.* Two robustness analyses were added and are reported as such: training on randomly permuted histories (Amazon, five mixers, three seeds) and embedding size 128 (Sports and MovieLens-1M, five mixers, three seeds). The popularity analysis uses the strata defined in the protocol; its pooled intervals were computed after the main analysis.

## APPENDIX B ADDITIONAL RESULTS

Table 7 lists the placebo results summarized in Fig. 3(a) and Section V-C.

**TABLE 7. Change of test NDCG@10 (%) when the test history is randomly permuted (perm.) or its most recent item is removed (−last), mean over five seeds.**

| | Baby | | Sports | | Clothing | | ML-1M | |
|---|---|---|---|---|---|---|---|---|
| Mixer | perm. | −last | perm. | −last | perm. | −last | perm. | −last |
| Last item | −48 | −37 | −45 | −38 | −40 | −38 | −83 | −30 |
| Last + mean | −27 | −33 | −21 | −27 | −18 | −25 | −76 | −26 |
| EMA | −40 | −37 | −35 | −34 | −30 | −33 | −80 | −24 |
| w/o sel. | −39 | −33 | −38 | −34 | −28 | −33 | −80 | −24 |
| Mamba | −39 | −31 | −36 | −34 | −30 | −33 | −80 | −25 |
| SASRec | −39 | −32 | −38 | −34 | −38 | −35 | −76 | −22 |
| GRU4Rec | −37 | −32 | −38 | −33 | −32 | −30 | −79 | −20 |

## APPENDIX C REPRODUCIBILITY DETAILS

Table 8 lists every fixed setting, and Table 9 the configuration selected on the validation set for every model and dataset. Table 10 maps the reproducibility items of [83] to this paper and its supplementary material.

## ACKNOWLEDGMENT

The authors used Claude (Anthropic) for code development, figure preparation, and drafting and editing of the text in all sections of this article. The authors reviewed and verified all content against the released result files and take full responsibility for the publication.

## REFERENCES

[1] M. Quadrana, P. Cremonesi, and D. Jannach, "Sequence-aware recommender systems," *ACM Comput. Surv.*, vol. 51, no. 4, pp. 1–36, 2019.
[2] S. Wang, L. Hu, Y. Wang, L. Cao, Q. Z. Sheng, and M. Orgun, "Sequential recommender systems: Challenges, progress and prospects," in *Proc. 28th Int. Joint Conf. Artif. Intell. (IJCAI)*, 2019, pp. 6332–6338.
[3] H. Fang, D. Zhang, Y. Shu, and G. Guo, "Deep learning for sequential recommendation: Algorithms, influential factors, and evaluations," *ACM Trans. Inf. Syst.*, vol. 39, no. 1, pp. 1–42, 2021.
[4] S. Rendle, C. Freudenthaler, and L. Schmidt-Thieme, "Factorizing personalized Markov chains for next-basket recommendation," in *Proc. 19th Int. Conf. World Wide Web (WWW)*, 2010, pp. 811–820.
[5] R. He and J. McAuley, "Fusing similarity models with Markov chains for sparse sequential recommendation," in *Proc. IEEE 16th Int. Conf. Data Mining (ICDM)*, 2016, pp. 191–200.
[6] B. Hidasi, A. Karatzoglou, L. Baltrunas, and D. Tikk, "Session-based recommendations with recurrent neural networks," in *Proc. 4th Int. Conf. Learn. Represent. (ICLR)*, 2016.
[7] J. Li, P. Ren, Z. Chen, Z. Ren, T. Lian, and J. Ma, "Neural attentive session-based recommendation," in *Proc. ACM Conf. Inf. Knowl. Manage. (CIKM)*, 2017, pp. 1419–1428.
[8] J. Tang and K. Wang, "Personalized top-N sequential recommendation via convolutional sequence embedding," in *Proc. 11th ACM Int. Conf. Web Search Data Mining (WSDM)*, 2018, pp. 565–573.
[9] W.-C. Kang and J. McAuley, "Self-attentive sequential recommendation," in *Proc. IEEE Int. Conf. Data Mining (ICDM)*, 2018, pp. 197–206.

**TABLE 8. Fixed settings and initializations (identical for all models unless stated).**

| Setting | Value |
|---|---|
| Embedding size $d$ | 64 (128 in the robustness check) |
| Blocks | 2 (GRU4Rec: one GRU layer) |
| Feed-forward network | inner size $4d$, GELU, dropout on the hidden layer |
| Dropout | selected from $\{0.2, 0.5\}$; embeddings, mixer and feed-forward outputs (SASRec: also attention weights) |
| Max. input length $L$ | 50 (Amazon), 200 (MovieLens-1M) |
| Mamba block | $2d$ channels, causal conv. width 4, $N = 16$, step-size rank $d/16$; $\Delta$ initialized log-uniformly in $[10^{-3}, 10^{-1}]$; $\mathbf{A}_c = -(1, \ldots, N)$ [41]; $D = 1$ |
| W/o selectivity | $\Delta$ per channel (same initialization); $\mathbf{B}, \mathbf{C} \in \mathbb{R}^N$ learned, initialized $\mathcal{N}(0, 0.25)$ |
| EMA | $\mathbf{a} = \sigma(\mathbf{w})$, $\mathbf{w}$ initialized to 1 ($a \approx 0.73$) |
| SASRec | 2 heads, learned positional embeddings |
| Item embeddings | $\mathcal{N}(0, 0.02^2)$, tied with the output layer |
| Optimizer | Adam, $\beta = (0.9, 0.999)$, $\epsilon = 10^{-8}$, no weight decay, no gradient clipping |
| Learning rate | selected from $\{10^{-3}, 3 \times 10^{-3}\}$ |
| Batch size | 256 users (128 for MovieLens-1M and Electronics) |
| Schedule | at most 200 epochs, validation every 2 epochs, patience 5 validations, best checkpoint |
| Loss | full-softmax cross-entropy at every position |
| Evaluation | full ranking, history items excluded, pessimistic ties |

**TABLE 9. Configuration selected on the validation set (learning rate / dropout) and mean best epoch over the five final seeds. Electronics uses the Sports configurations.**

| Mixer | Baby | Sports | Clothing | MovieLens-1M |
|---|---|---|---|---|
| Last item | 1e-3/0.2 (10) | 1e-3/0.2 (8) | 1e-3/0.2 (9) | 1e-3/0.2 (44) |
| Last + mean | 1e-3/0.2 (15) | 1e-3/0.5 (16) | 1e-3/0.5 (16) | 3e-3/0.2 (37) |
| EMA | 1e-3/0.2 (14) | 1e-3/0.2 (8) | 1e-3/0.2 (8) | 1e-3/0.2 (48) |
| w/o sel. | 1e-3/0.5 (21) | 1e-3/0.2 (10) | 3e-3/0.5 (25) | 1e-3/0.2 (62) |
| Mamba | 1e-3/0.2 (10) | 1e-3/0.5 (18) | 3e-3/0.5 (39) | 3e-3/0.2 (40) |
| SASRec | 1e-3/0.2 (11) | 1e-3/0.2 (8) | 1e-3/0.5 (15) | 3e-3/0.2 (43) |
| GRU4Rec | 1e-3/0.5 (42) | 1e-3/0.2 (30) | 3e-3/0.2 (9) | 3e-3/0.5 (64) |

**TABLE 10. Reproducibility items and where they are addressed.**

| Item | Location |
|---|---|
| Model and mixer definitions | Section III, Table 8 |
| Data sources, preprocessing, splits, statistics | Section IV, Table 1; split-check and MovieLens scripts with SHA-256 hashes |
| Search space, selection criterion, selected values | Section IV, Tables 8 and 9 |
| Number of runs and seeds | Section IV-G |
| Metrics and evaluation protocol | Section IV |
| Statistical procedure, intervals, correction | Section IV, Table 3 |
| Hardware, software, run time | Section IV-G, Table 6 |
| Pre-registration and deviations | Appendix A; protocol file |
| Code and per-run results (incl. per-user ranks) | code repository (Section IV-G) and supplementary material |

[10] F. Sun *et al.*, "BERT4Rec: Sequential recommendation with bidirectional encoder representations from transformer," in *Proc. 28th ACM Int. Conf. Inf. Knowl. Manage. (CIKM)*, 2019, pp. 1441–1450.

[11] A. Gu and T. Dao, "Mamba: Linear-time sequence modeling with selective state spaces," in *Proc. 1st Conf. Lang. Model. (COLM)*, 2024.

[12] C. Liu, J. Lin, J. Wang, H. Liu, and J. Caverlee, "Mamba4Rec: Towards efficient sequential recommendation with selective state space models," arXiv:2403.03900, 2024.

[13] Z. Liu *et al.*, "SIGMA: Selective gated Mamba for sequential recommendation," in *Proc. AAAI Conf. Artif. Intell.*, vol. 39, no. 12, 2025, pp. 12 264–12 272.

[14] Y. Zhang, H. Qu, L. Ning, W. Fan, and Q. Li, "SSD4Rec: A structured state space duality model for efficient sequential recommendation," *ACM Trans. Inf. Syst.*, vol. 44, no. 2, pp. 1–26, 2026.

[15] H. Fan *et al.*, "TiM4Rec: An efficient sequential recommendation model based on time-aware structured state space duality model," *Neurocomputing*, vol. 654, p. 131270, 2025.

[16] Z. Zhang, B. Yang, and Y. Lu, "A local context enhanced consistency-aware Mamba-based sequential recommendation model," *Inf. Process. Manage.*, vol. 62, no. 3, p. 104076, 2025.

[17] A. Klenitskiy and A. Vasilev, "Turning dross into gold loss: Is BERT4Rec really better than SASRec?" in *Proc. 17th ACM Conf. Recommender Syst. (RecSys)*, 2023, pp. 1120–1125.

[18] A. V. Petrov and C. Macdonald, "gSASRec: Reducing overconfidence in sequential recommendation trained with negative sampling," in *Proc. 17th ACM Conf. Recommender Syst. (RecSys)*, 2023, pp. 116–128.

[19] B. Hidasi and Á. T. Czapp, "The effect of third party implementations on reproducibility," in *Proc. 17th ACM Conf. Recommender Syst. (RecSys)*, 2023, pp. 272–282.

[20] M. Ferrari Dacrema, P. Cremonesi, and D. Jannach, "Are we really making much progress? A worrying analysis of recent neural recommendation approaches," in *Proc. 13th ACM Conf. Recommender Syst. (RecSys)*, 2019, pp. 101–109.

[21] S. Rendle, W. Krichene, L. Zhang, and J. Anderson, "Neural collaborative filtering vs. matrix factorization revisited," in *Proc. 14th ACM Conf. Recommender Syst. (RecSys)*, 2020, pp. 240–248.

[22] M. Ludewig, N. Mauro, S. Latifi, and D. Jannach, "Empirical analysis of session-based recommendation algorithms," *User Model. User-Adapted Interact.*, vol. 31, no. 1, pp. 149–181, 2021.

[23] A. Klenitskiy, A. Volodkevich, A. Pembek, and A. Vasilev, "Does it look sequential? An analysis of datasets for evaluation of sequential recommendations," in *Proc. 18th ACM Conf. Recommender Syst. (RecSys)*, 2024, pp. 1067–1072.

[24] B. Hidasi and A. Karatzoglou, "Recurrent neural networks with top-k gains for session-based recommendations," in *Proc. 27th ACM Int. Conf. Inf. Knowl. Manage. (CIKM)*, 2018, pp. 843–852.

[25] J. Li, Y. Wang, and J. McAuley, "Time interval aware self-attention for sequential recommendation," in *Proc. 13th Int. Conf. Web Search Data Mining (WSDM)*, 2020, pp. 322–330.

[26] Q. Liu, Y. Zeng, R. Mokhosi, and H. Zhang, "STAMP: Short-term attention/memory priority model for session-based recommendation," in *Proc. 24th ACM SIGKDD Int. Conf. Knowl. Discovery Data Mining (KDD)*, 2018, pp. 1831–1839.

[27] X. Fan, Z. Liu, J. Lian, W. X. Zhao, X. Xie, and J.-R. Wen, "Lighter and better: Low-rank decomposed self-attention networks for next-item recommendation," in *Proc. 44th Int. ACM SIGIR Conf. Res. Develop. Inf. Retr.*, 2021, pp. 1733–1737.

[28] L. Liu *et al.*, "LinRec: Linear attention mechanism for long-term sequential recommender systems," in *Proc. 46th Int. ACM SIGIR Conf. Res. Develop. Inf. Retr.*, 2023, pp. 289–299.

[29] K. Zhou, H. Yu, W. X. Zhao, and J.-R. Wen, "Filter-enhanced MLP is all you need for sequential recommendation," in *Proc. ACM Web Conf. (WWW)*, 2022, pp. 2388–2399.

[30] Z. Yue, Y. Wang, Z. He, H. Zeng, J. McAuley, and D. Wang, "Linear recurrent units for sequential recommendation," in *Proc. 17th ACM Int. Conf. Web Search Data Mining (WSDM)*, 2024, pp. 930–938.

[31] C. Liu, J. Lin, H. Liu, J. Wang, and J. Caverlee, "Behavior-dependent linear recurrent units for efficient sequential recommendation," in *Proc. 33rd ACM Int. Conf. Inf. Knowl. Manage. (CIKM)*, 2024, pp. 1430–1440.

[32] Y. Ding and X. Li, "Time weight collaborative filtering," in *Proc. 14th ACM Int. Conf. Inf. Knowl. Manage. (CIKM)*, 2005, pp. 485–492.

[33] Y. Koren, "Collaborative filtering with temporal dynamics," in *Proc. 15th ACM SIGKDD Int. Conf. Knowl. Discovery Data Mining (KDD)*, 2009, pp. 447–456.

[34] A. Anderson, R. Kumar, A. Tomkins, and S. Vassilvitskii, "The dynamics of repeat consumption," in *Proc. 23rd Int. Conf. World Wide Web (WWW)*, 2014, pp. 419–430.

[35] M. Ludewig and D. Jannach, "Evaluation of session-based recommendation algorithms," *User Model. User-Adapted Interact.*, vol. 28, no. 4–5, pp. 331–390, 2018.

[36] D. Garg, P. Gupta, P. Malhotra, L. Vig, and G. Shroff, "Sequence and time aware neighborhood for session-based recommendations: STAN," in *Proc. 42nd Int. ACM SIGIR Conf. Res. Develop. Inf. Retr.*, 2019, pp. 1069–1072.

[37] M. Ludewig, N. Mauro, S. Latifi, and D. Jannach, "Performance comparison of neural and non-neural approaches to session-based recommendation," in *Proc. 13th ACM Conf. Recommender Syst. (RecSys)*, 2019, pp. 462–466.

[38] S. Latifi, N. Mauro, and D. Jannach, "Session-aware recommendation: A surprising quest for the state-of-the-art," *Inf. Sci.*, vol. 573, pp. 291–315, 2021.

[39] A. Gu, T. Dao, S. Ermon, A. Rudra, and C. Ré, "HiPPO: Recurrent memory with optimal polynomial projections," in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, vol. 33, 2020, pp. 1474–1487.

[40] A. Gu, K. Goel, and C. Ré, "Efficiently modeling long sequences with structured state spaces," in *Proc. 10th Int. Conf. Learn. Represent. (ICLR)*, 2022.

[41] A. Gu, K. Goel, A. Gupta, and C. Ré, "On the parameterization and initialization of diagonal state space models," in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, vol. 35, 2022, pp. 35 971–35 983.

[42] J. T. H. Smith, A. Warrington, and S. W. Linderman, "Simplified state space layers for sequence modeling," in *Proc. 11th Int. Conf. Learn. Represent. (ICLR)*, 2023.

[43] A. Orvieto *et al.*, "Resurrecting recurrent neural networks for long sequences," in *Proc. 40th Int. Conf. Mach. Learn. (ICML)*, ser. Proc. Mach. Learn. Res., vol. 202, 2023, pp. 26 670–26 698.

[44] T. Dao and A. Gu, "Transformers are SSMs: Generalized models and efficient algorithms through structured state space duality," in *Proc. 41st Int. Conf. Mach. Learn. (ICML)*, ser. Proc. Mach. Learn. Res., vol. 235, 2024, pp. 10 041–10 071.

[45] H. Qu *et al.*, "A survey of Mamba," *ACM Trans. Intell. Syst. Technol.*, vol. 17, no. 5, pp. 1–43, 2026.

[46] B. Peng *et al.*, "RWKV: Reinventing RNNs for the transformer era," in *Findings Assoc. Comput. Linguistics: EMNLP*, 2023, pp. 14 048–14 077.

[47] S. Yang, B. Wang, Y. Shen, R. Panda, and Y. Kim, "Gated linear attention transformers with hardware-efficient training," in *Proc. 41st Int. Conf. Mach. Learn. (ICML)*, ser. Proc. Mach. Learn. Res., vol. 235, 2024, pp. 56 501–56 523.

[48] X. Ma *et al.*, "Mega: Moving average equipped gated attention," in *Proc. 11th Int. Conf. Learn. Represent. (ICLR)*, 2023.

[49] S. Arora *et al.*, "Zoology: Measuring and improving recall in efficient language models," in *Proc. 12th Int. Conf. Learn. Represent. (ICLR)*, 2024.

[50] S. Jelassi, D. Brandfonbrener, S. M. Kakade, and E. Malach, "Repeat after me: Transformers are better than state space models at copying," in *Proc. 41st Int. Conf. Mach. Learn. (ICML)*, ser. Proc. Mach. Learn. Res., vol. 235, 2024, pp. 21 502–21 521.

[51] A. Ali, I. Zimerman, and L. Wolf, "The hidden attention of Mamba models," in *Proc. 63rd Annu. Meeting Assoc. Comput. Linguistics (ACL)*, 2025, pp. 1516–1534.

[52] M. C. Airlangga, H. AlQuabeh, M. Nwadike, and K. Inui, "Emergence of primacy and recency effect in Mamba: A mechanistic point of view," arXiv:2506.15156, 2025.

[53] D. Han *et al.*, "Demystify Mamba in vision: A linear attention perspective," in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, vol. 37, 2024, pp. 127 181–127 203.

[54] W. Yu and X. Wang, "MambaOut: Do we really need Mamba for vision?" in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2025, pp. 4484–4496.

[55] Z. Wang *et al.*, "Is Mamba effective for time series forecasting?" *Neurocomputing*, vol. 619, p. 129178, 2025.

[56] J. Sieber, C. Amo Alonso, A. Didier, M. N. Zeilinger, and A. Orvieto, "Understanding the differences in foundation models: Attention, state space models, and recurrent neural networks," in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, vol. 37, 2024.

[57] M. Ferrari Dacrema, S. Boglio, P. Cremonesi, and D. Jannach, "A troubling analysis of reproducibility and progress in recommender systems research," *ACM Trans. Inf. Syst.*, vol. 39, no. 2, pp. 1–49, 2021.

[58] D. Jannach and M. Ludewig, "When recurrent neural networks meet the neighborhood for session-based recommendation," in *Proc. 11th ACM Conf. Recommender Syst. (RecSys)*, 2017, pp. 306–310.

[59] Z. Sun *et al.*, "Are we evaluating rigorously? Benchmarking recommendation for reproducible evaluation and fair comparison," in *Proc. 14th ACM Conf. Recommender Syst. (RecSys)*, 2020, pp. 23–32.

[60] A. Bellogín and A. Said, "Improving accountability in recommender systems research through reproducibility," *User Model. User-Adapted Interact.*, vol. 31, no. 5, pp. 941–977, 2021.

[61] A. Petrov and C. Macdonald, "A systematic review and replicability study of BERT4Rec for sequential recommendation," in *Proc. 16th ACM Conf. Recommender Syst. (RecSys)*, 2022, pp. 436–447.

[62] W. Krichene and S. Rendle, "On sampled metrics for item recommendation," in *Proc. 26th ACM SIGKDD Int. Conf. Knowl. Discovery Data Mining (KDD)*, 2020, pp. 1748–1757.

[63] A. Dallmann, D. Zoller, and A. Hotho, "A case study on sampling strategies for evaluating neural sequential item recommendation models," in *Proc. 15th ACM Conf. Recommender Syst. (RecSys)*, 2021, pp. 505–514.

[64] Z. Meng, R. McCreadie, C. Macdonald, and I. Ounis, "Exploring data splitting strategies for the evaluation of recommendation models," in *Proc. 14th ACM Conf. Recommender Syst. (RecSys)*, 2020, pp. 681–686.

[65] Y. Ji, A. Sun, J. Zhang, and C. Li, "A critical study on data leakage in recommender system offline evaluation," *ACM Trans. Inf. Syst.*, vol. 41, no. 3, pp. 1–27, 2023.

[66] T. Sakai, "Statistical reform in information retrieval?" *ACM SIGIR Forum*, vol. 48, no. 1, pp. 3–12, 2014.

[67] B. A. Carterette, "Multiple testing in statistical analysis of systems-based information retrieval experiments," *ACM Trans. Inf. Syst.*, vol. 30, no. 1, pp. 1–34, 2012.

[68] R. Dror, G. Baumer, S. Shlomov, and R. Reichart, "The hitchhiker's guide to testing statistical significance in natural language processing," in *Proc. 56th Annu. Meeting Assoc. Comput. Linguistics (ACL)*, 2018, pp. 1383–1392.

[69] X. Bouthillier *et al.*, "Accounting for variance in machine learning benchmarks," in *Proc. Mach. Learn. Syst. (MLSys)*, vol. 3, 2021, pp. 747–769.

[70] J. L. Ba, J. R. Kiros, and G. E. Hinton, "Layer normalization," arXiv:1607.06450, 2016.

[71] J. McAuley, C. Targett, Q. Shi, and A. van den Hengel, "Image-based recommendations on styles and substitutes," in *Proc. 38th Int. ACM SIGIR Conf. Res. Develop. Inf. Retr.*, 2015, pp. 43–52.

[72] R. He and J. McAuley, "Ups and downs: Modeling the visual evolution of fashion trends with one-class collaborative filtering," in *Proc. 25th Int. Conf. World Wide Web (WWW)*, 2016, pp. 507–517.

[73] X. Zhou, "MMRec: Simplifying multimodal recommendation," in *Proc. 5th ACM Int. Conf. Multimedia in Asia Workshops*, 2023, pp. 1–2.

[74] F. M. Harper and J. A. Konstan, "The MovieLens datasets: History and context," *ACM Trans. Interact. Intell. Syst.*, vol. 5, no. 4, pp. 1–19, 2015.

[75] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," in *Proc. 3rd Int. Conf. Learn. Represent. (ICLR)*, 2015.

[76] E. Walker and A. S. Nowacki, "Understanding equivalence and noninferiority testing," *J. Gen. Intern. Med.*, vol. 26, no. 2, pp. 192–196, 2011.

[77] B. Efron and R. J. Tibshirani, *An Introduction to the Bootstrap*. New York, NY, USA: Chapman & Hall, 1993.

[78] F. Wilcoxon, "Individual comparisons by ranking methods," *Biometrics Bull.*, vol. 1, no. 6, pp. 80–83, 1945.

[79] S. Holm, "A simple sequentially rejective multiple test procedure," *Scand. J. Statist.*, vol. 6, no. 2, pp. 65–70, 1979.

[80] M. D. Smucker, J. Allan, and B. Carterette, "A comparison of statistical significance tests for information retrieval evaluation," in *Proc. 16th ACM Conf. Inf. Knowl. Manage. (CIKM)*, 2007, pp. 623–632.

[81] B. A. Nosek, C. R. Ebersole, A. C. DeHaven, and D. T. Mellor, "The preregistration revolution," *Proc. Nat. Acad. Sci. USA*, vol. 115, no. 11, pp. 2600–2606, 2018.

[82] W. X. Zhao *et al.*, "RecBole: Towards a unified, comprehensive and efficient framework for recommendation algorithms," in *Proc. 30th ACM Int. Conf. Inf. Knowl. Manage. (CIKM)*, 2021, pp. 4653–4664.

[83] J. Pineau *et al.*, "Improving reproducibility in machine learning research (a report from the NeurIPS 2019 reproducibility program)," *J. Mach. Learn. Res.*, vol. 22, no. 164, pp. 1–20, 2021.